

✓ ITAI 1371 Midterm: Netflix Movies Data Preparation

Project: Predict whether a Netflix *movie* is kid-safe (`kid_safe`) from its metadata.

Dataset: Kaggle `shivamb/netflix-shows` (movies only). **Requirement:** the raw file was not edited by hand. Changes were produced using Python.

Pipeline order (and why)

1. Load the raw data and keep movies only
2. Inspect raw data: missing values, odd values
3. Build the `kid_safe` target from `rating`
4. Engineer features (row-by-row, so there is no leakage)
5. **Split 70/30 before anything is learned from the data**
6. Fill missing values (statistics learned from the training set only)
7. Normalize skewed numbers, then scale ranges
8. Encode categories (one-hot, multi-hot, ordinal)
9. Balance classes (training set only)
10. Save the outputs and discuss the ML problem

```
# Step 0: setup
import warnings, os, glob
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PowerTransformer, MinMaxScaler, One
from sklearn.utils import resample

warnings.filterwarnings("ignore")
pd.set_option("display.max_columns", 50, "display.width", 200)
RANDOM_STATE = 42
plt.rcParams["figure.dpi"] = 100
```

✓ Step 1. Load the raw data and keep movies only

```
# Download the public Kaggle dataset in code (no manual download or ex
# If kagglehub is unavailable, upload netflix_titles.csv to the Colab
```

```

try:
    import kagglehub
    path = kagglehub.dataset_download("shivamb/netflix-shows")
    csv_path = glob.glob(os.path.join(path, "*.csv"))[0]
except Exception as e:
    print("kagglehub failed, falling back to a local file:", e)
    csv_path = "netflix_titles.csv"

raw = pd.read_csv(csv_path)
print("Source file:", csv_path)
print("Raw shape (movies + TV shows):", raw.shape)
print(raw["type"].value_counts())
raw.head()

```

Using Colab cache for faster access to the 'netflix-shows' dataset.

Source file: /kaggle/input/netflix-shows/netflix_titles.csv

Raw shape (movies + TV shows): (8807, 12)

type

Movie 6131

TV Show 2676

Name: count, dtype: int64

	show_id	type	title	director	cast	country	date_added	rel
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States	September 25, 2021	
1	s2	TV Show	Blood & Water	NaN	Ama Qamata, Khosi Ngema, Gail Mabalane, Thaban...	South Africa	September 24, 2021	
2	s3	TV Show	Ganglands	Julien Leclercq	Sami Bouajila, Tracy Gotoas, Samuel Jouy, Nabi...	NaN	September 24, 2021	
3	s4	TV Show	Jailbirds New Orleans	NaN	NaN	NaN	September 24, 2021	

Next steps:

[Generate code with raw](#)

[New interactive sheet](#)

```
# Keep movies only (TV shows removed in code)
movies = raw[raw["type"].str.strip() == "Movie"].copy()
print(f"Rows before filter: {len(raw):,} | Movies kept: {len(movies):,}")
assert len(movies) >= 2000, "Dataset must have at least 2,000 rows"
print("Meets the 2,000-row minimum:", len(movies) >= 2000)
movies.head()
```

Rows before filter: 8,807 | Movies kept: 6,131 | TV shows removed: 2,676
Meets the 2,000-row minimum: True

	show_id	type	title	director	cast	country	date_added
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States	September 25, 2021
6	s7	Movie	My Little Pony: A New Generation	Robert Cullen, José Luis Ucha	Vanessa Hudgens, Kimiko Glenn, James Marsden, ...	NaN	September 24, 2021
7	s8	Movie	Sankofa	Haile Gerima	Kofi Ghanaba, Oyafunmike Ogunlano, Alexandra D...	United States, Ghana, Burkina Faso, United Kin...	September 24, 2021
9	s10	Movie	The Starling	Theodore Melfi	Melissa McCarthy, Chris O'Dowd, Kevin Kline, T...	United States	September 24, 2021

Next steps: [Generate code with movies](#) [New interactive sheet](#)

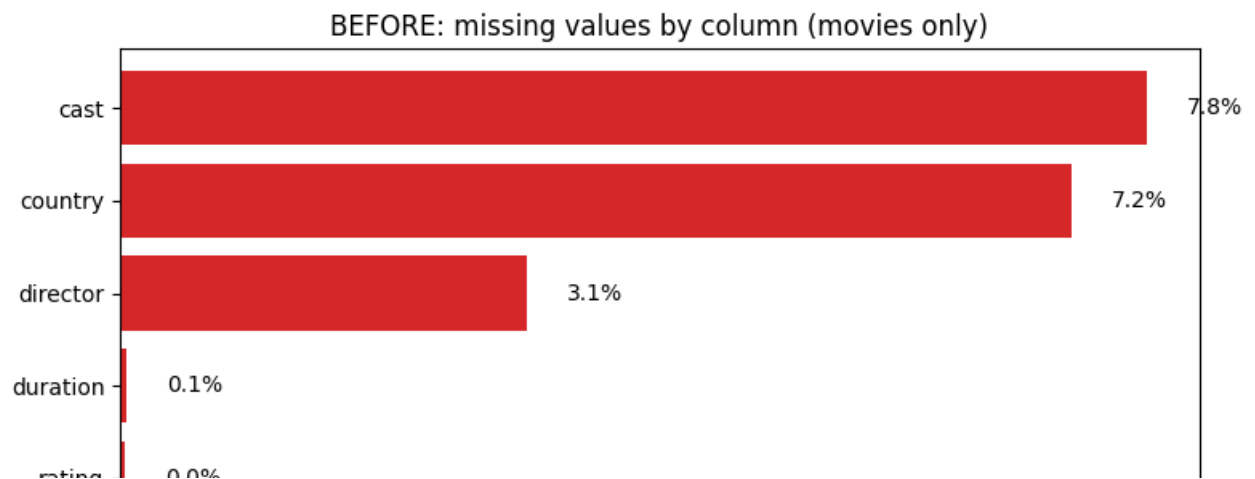
✓ Step 2. Inspect the raw movie data (BEFORE cleanup)

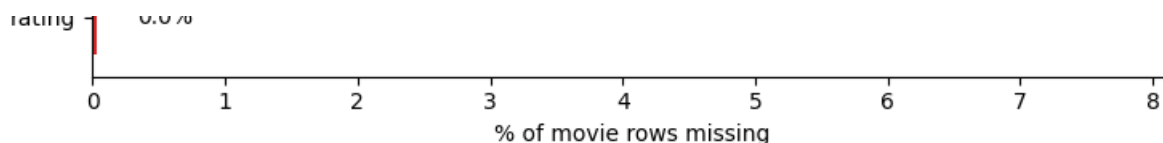
```
print(movies.dtypes, "\n")
missing_raw = movies.isna().sum()
missing_pct = (missing_raw / len(movies) * 100).round(2)
missing_table = pd.DataFrame({"missing": missing_raw, "percent": missing_pct})
print(missing_table)
```

```
fig, ax = plt.subplots(figsize=(8, 4))
shown = missing_table[missing_table["missing"] > 0]
ax.barh(shown.index[::-1], shown["percent"][:,::-1], color="tab:red")
ax.set_xlabel("% of movie rows missing")
ax.set_title("BEFORE: missing values by column (movies only)")
for i, v in enumerate(shown["percent"][:,::-1]):
    ax.text(v + 0.3, i, f"{v:.1f}%", va="center")
plt.tight_layout(); plt.show()
```

```
show_id      object
type         object
title        object
director     object
cast         object
country      object
date_added   object
release_year  int64
rating       object
duration     object
listed_in    object
description  object
dtype: object
```

	missing	percent
cast	475	7.75
country	440	7.18
director	188	3.07
duration	3	0.05
rating	2	0.03
title	0	0.00
show_id	0	0.00
type	0	0.00
release_year	0	0.00
date_added	0	0.00
listed_in	0	0.00
description	0	0.00





```
# Look for values that are present but wrong. In this dataset a few movies
# typed into the 'rating' column and an empty 'duration'.
movies["rating"] = movies["rating"].astype("string").str.strip()
misplaced = movies["rating"].str.contains("min", na=False)
print("Rows with a run time stored in 'rating':", misplaced.sum())
print(movies.loc[misplaced, ["title", "rating", "duration"]])
```

```
# Fix in code: move the value to duration, and treat the rating as missing
movies.loc[misplaced, "duration"] = movies.loc[misplaced, "rating"]
movies.loc[misplaced, "rating"] = pd.NA
```

```
print("\nRating counts after the fix:")
print(movies["rating"].value_counts(dropna=False))
```

```
Rows with a run time stored in 'rating': 0
Empty DataFrame
Columns: [title, rating, duration]
Index: []
```

```
Rating counts after the fix:
```

```
rating
TV-MA      2062
TV-14      1427
R           797
TV-PG       540
PG-13       490
PG          287
TV-Y7       139
TV-Y        131
TV-G        126
NR           75
G           41
TV-Y7-FV     5
<NA>         5
NC-17         3
UR            3
Name: count, dtype: Int64
```

✓ Step 3. Build the `kid_safe` target from `rating`

Decision (documented):

- `kid_safe = 1` for G, PG, TV-G, TV-PG, TV-Y, TV-Y7, TV-Y7-FV
- `kid_safe = 0` for PG-13, R, NC-17, TV-14, TV-MA
- `NR`, `UR`, and missing ratings have **no known maturity label**. They are described as "unrated" and are **not** treated as statistical outliers. They are excluded from the modeling table because we cannot assign them a true label.

The `rating` column is used only to build the target and is **not** a model input.

```
KID_SAFE = {"G", "PG", "TV-G", "TV-PG", "TV-Y", "TV-Y7", "TV-Y7-FV"}
MATURE = {"PG-13", "R", "NC-17", "TV-14", "TV-MA"}

def make_target(r):
    if pd.isna(r): return np.nan
    if r in KID_SAFE: return 1
    if r in MATURE: return 0
    return np.nan    # NR, UR, or anything unrecognised

movies["kid_safe"] = movies["rating"].map(make_target)

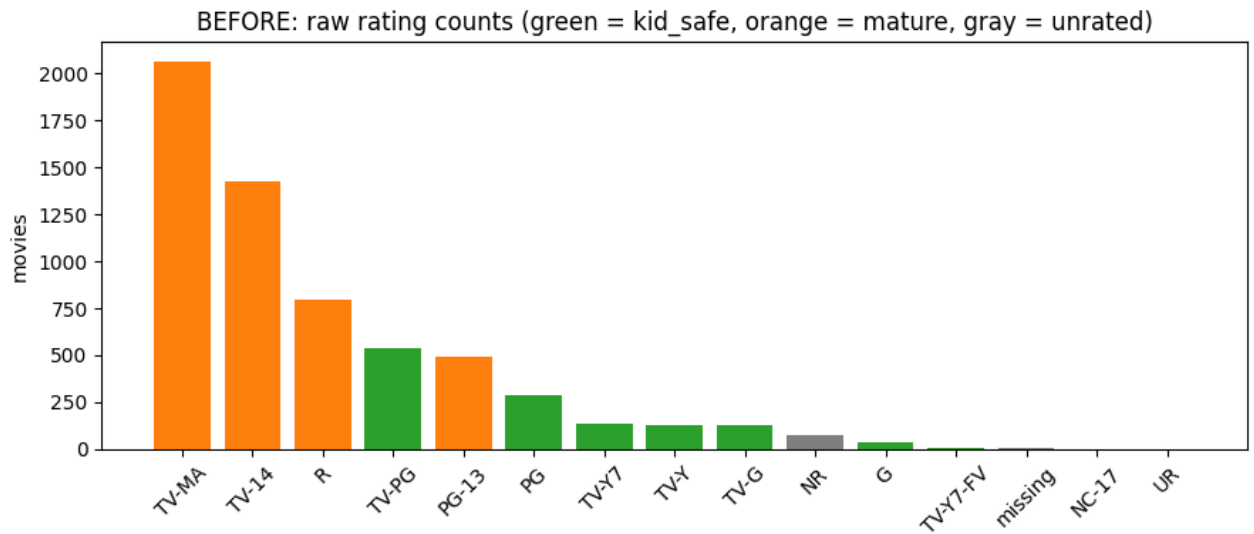
unrated = movies["kid_safe"].isna()
print("Unrated / unknown-rating movies:", unrated.sum())
print(movies.loc[unrated, "rating"].value_counts(dropna=False), "\n")

fig, ax = plt.subplots(figsize=(9, 4))
order = movies["rating"].value_counts(dropna=False)
labels_r = [str(r) if pd.notna(r) else "missing" for r in order.index]
colors = ["tab:green" if r in KID_SAFE else "tab:orange" if r in MATURE else "tab:gray" for r in order.index]
ax.bar(labels_r, order.values, color=colors)
ax.set_title("BEFORE: raw rating counts (green = kid_safe, orange = mature, gray = unrated)")
ax.set_ylabel("movies"); plt.xticks(rotation=45); plt.tight_layout();

model_df = movies.loc[~unrated].copy()
model_df["kid_safe"] = model_df["kid_safe"].astype(int)
print(f"Movies kept for modeling: {len(model_df):,} (dropped {unrated.sum():,} unrated movies)")
print(model_df["kid_safe"].value_counts().rename({1: "kid_safe", 0: "mature", 0: "unrated"}))
```

```
Unrated / unknown-rating movies: 83
rating
NR      75
<NA>    5
TV-14    2
```

```
UR
J
Name: count, dtype: Int64
```



```
Movies kept for modeling: 6,048 (dropped 83 unrated)
kid_safe
mature      4779
kid_safe    1269
Name: count, dtype: int64
```

✓ Step 4. Feature engineering

These features are computed row by row from a single movie's own information, so they cannot leak information between train and test. Missing text fields get a clear `*_missing` flag so the missingness itself is documented and available to a model.

```
def count_items(s):
    return s.fillna("").apply(lambda x: len([i for i in x.split(",")])

df = model_df.copy()
df["duration_min"] = df["duration"].str.extract(r"(\d+)")[0].astype(f

added = pd.to_datetime(df["date_added"].astype("string").str.strip(),
df["year_added"] = added.dt.year
df["month_added"] = added.dt.month
```

```

df["years_to_netflix"] = (df["year_added"] - df["release_year"]).clip(
    lower=0, upper=100)

df["n_cast"] = count_items(df["cast"])
df["n_directors"] = count_items(df["director"])
df["n_countries"] = count_items(df["country"])
df["n_genres"] = count_items(df["listed_in"])
df["director_missing"] = df["director"].isna().astype(int)
df["cast_missing"] = df["cast"].isna().astype(int)
df["country_missing"] = df["country"].isna().astype(int)

df["desc_words"] = df["description"].fillna("").str.split().str.len()
df["title_len"] = df["title"].fillna("").str.len()

df["primary_country"] = df["country"].str.split(",").str[0].str.strip()
df["genres"] = df["listed_in"].fillna("").apply(lambda x: [g.strip()

num_cols = ["release_year", "duration_min", "year_added", "month_added",
            "n_cast", "n_directors", "n_countries", "n_genres", "desc_
flag_cols = ["director_missing", "cast_missing", "country_missing"]

print("Engineered columns:", ["duration_min", "year_added", "month_adde
            "n_cast", "n_directors", "n_countries", "n_genres", "desc_words",
            "primary_country", "genres", "*_missing flags"])
df[["title", "duration", "duration_min", "date_added", "year_added", '

```

```
Engineered columns: ['duration_min', 'year_added', 'month_added', 'year_
```

	title	duration	duration_min	date_added	year_added	years_to_
0	Dick Johnson Is Dead	90 min	90.0	September 25, 2021	2021	
6	My Little Pony: A New Generation	91 min	91.0	September 24, 2021	2021	
7	Sankofa	125 min	125.0	September 24, 2021	2021	
9	The Starling	104 min	104.0	September 24, 2021	2021	
12	Je Suis Karl	127 min	127.0	September 23, 2021	2021	
13	Confessions of an Invisible Girl	91 min	91.0	September 22, 2021	2021	
	Europe's					

Step 5. Train/test split (70/30, stratified)

We split **before** filling, scaling, encoding, or balancing so that nothing is learned from the test set. The test set stays untouched and keeps its natural class mix, because it represents unseen data.

```
feature_cols = num_cols + flag_cols + ["primary_country", "genres"]
X = df[feature_cols]
y = df["kid_safe"]
assert "rating" not in X.columns and "kid_safe" not in X.columns

Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.30, stratify=y)
print(f"Train: {len(Xtr):,} rows | Test: {len(Xte):,} rows")
print("Train kid_safe share:", round(ytr.mean(), 3), "| Test kid_safe share:", round(yte.mean(), 3))

Train: 4,233 rows | Test: 1,815 rows
Train kid_safe share: 0.21 | Test kid_safe share: 0.21
```

Step 6. Fill missing values (statistics learned from the training set only)

```
before_missing = Xtr.isna().sum()
print("Missing values in TRAIN before filling:")
print(before_missing[before_missing > 0], "\n")
print("Sample rows with a missing value (BEFORE):")
display(Xtr[Xtr[num_cols + ["primary_country"]].isna().any(axis=1)].head())

medians = Xtr[num_cols].median() # learned from train only
Xtr_i, Xte_i = Xtr.copy(), Xte.copy()
for d in (Xtr_i, Xte_i):
    d[num_cols] = d[num_cols].fillna(medians)
    d["primary_country"] = d["primary_country"].fillna("Unknown")

after_missing = Xtr_i.isna().sum()
print("\nMissing values in TRAIN after filling (should all be 0):")
print(after_missing[after_missing > 0] if after_missing.sum() else "no missing values")

cols = before_missing[before_missing > 0].index.tolist()
```

```

if cols:
    fig, ax = plt.subplots(figsize=(8, 4))
    w = 0.4; idx = np.arange(len(cols))
    ax.bar(idx - w/2, before_missing[cols], w, label="before", color='r')
    ax.bar(idx + w/2, after_missing[cols], w, label="after", color='g')
    ax.set_xticks(idx); ax.set_xticklabels(cols, rotation=30); ax.set_
    ax.set_title("Missing values before vs after filling"); ax.legend

print("\nSample rows (AFTER):")
display(Xtr_i[num_cols[:5] + ["primary_country"]].head(5))

```

Missing values in TRAIN before filling:

```

primary_country    293
dtype: int64

```

Sample rows with a missing value (BEFORE):

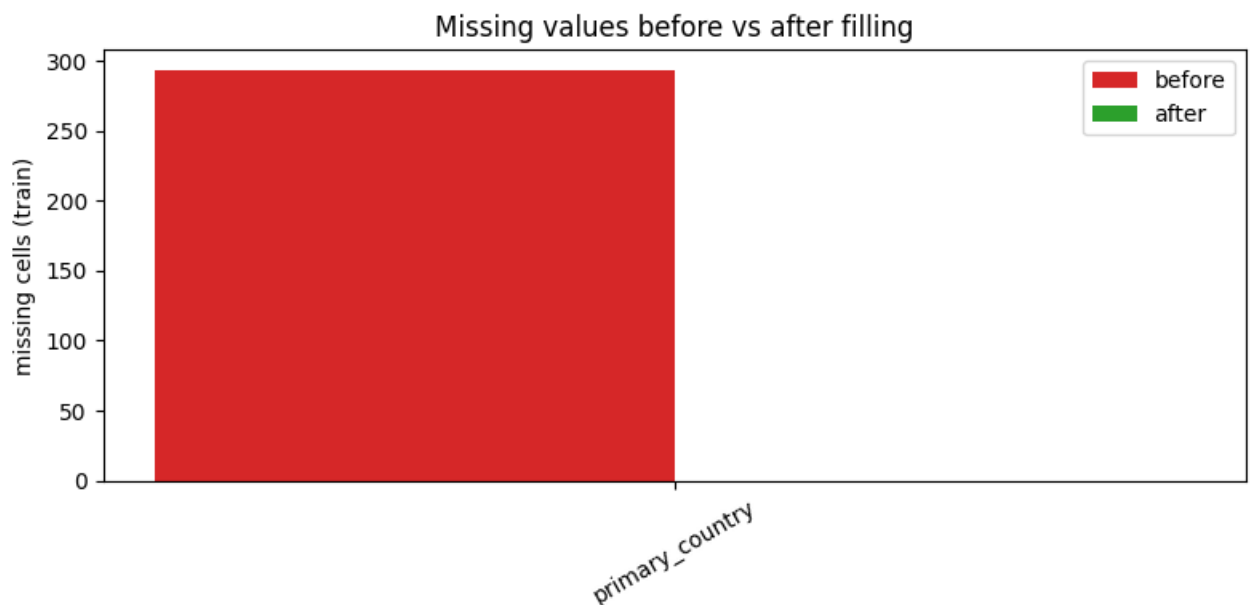
	release_year	duration_min	year_added	month_added	years_to_netf
6679	2016	79.0	2017	10	
1289	2020	58.0	2021	2	
421	2014	63.0	2021	7	
938	2019	87.0	2021	5	
7146	2017	52.0	2018	9	

Missing values in TRAIN after filling (should all be 0):

```

none

```



Sample rows (AFTER):

	release_year	duration_min	year_added	month_added	years_to_netf
--	--------------	--------------	------------	-------------	---------------

7653	2016	107.0	2018	5
2646	2016	107.0	2020	4
3044	2020	102.0	2020	1
8635	1969	128.0	2020	1
7516	2016	105.0	2018	12

Decisions (documented): numeric gaps are filled with the *training median* (robust to skew). Missing `primary_country` becomes its own category, `"Unknown"`, because missing country is information, not a random gap. Director and cast gaps are already captured by the `*_missing` flags and `n_*` counts (0 people listed).

✓ Step 7. Outliers, normalization (shape), and scaling (range)

Unusually long or short movies and old release years are **real data, not errors**, so they are kept. We check them with the IQR rule, and then reduce skew and put every number on a 0 to 1 range.

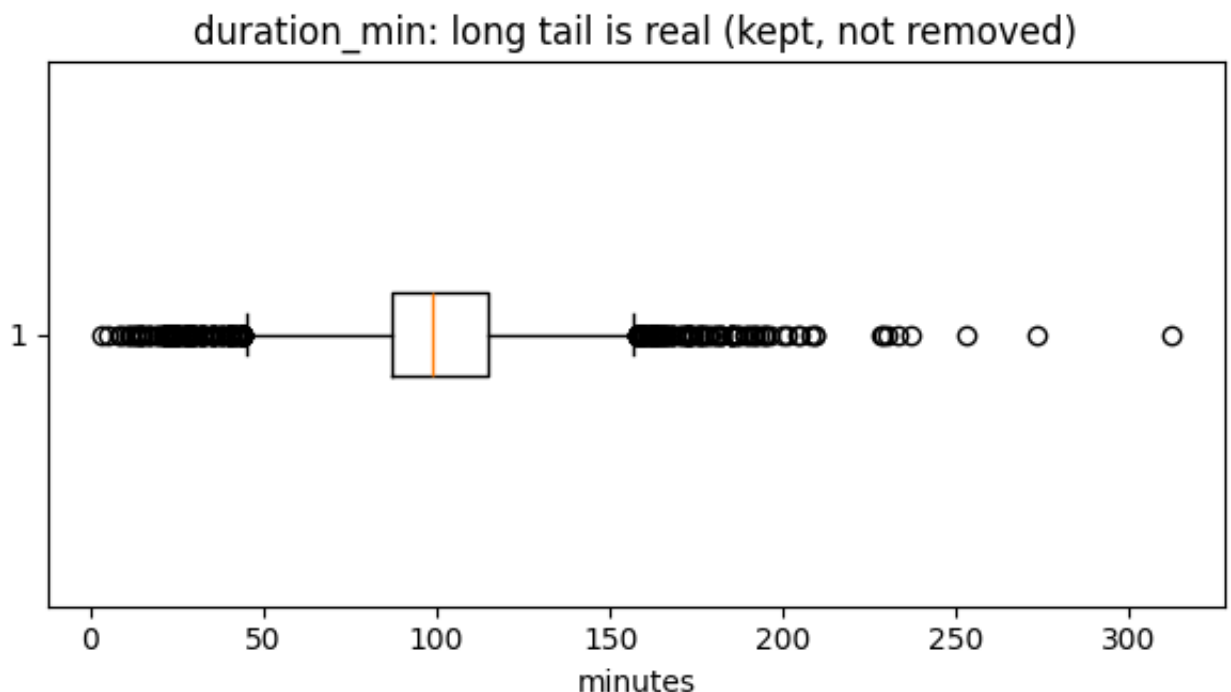
```

q1, q3 = Xtr_i[num_cols].quantile(0.25), Xtr_i[num_cols].quantile(0.75)
iqr = q3 - q1
outlier_counts = ((Xtr_i[num_cols] < q1 - 1.5 * iqr) | (Xtr_i[num_cols] > q3 + 1.5 * iqr)).sum()
skew = Xtr_i[num_cols].skew().round(2)
summary = pd.DataFrame({"skew": skew, "iqr_outliers": outlier_counts, "min": min, "max": max})
print(summary.sort_values("skew", key=abs, ascending=False))

fig, ax = plt.subplots(figsize=(6, 3.5))
ax.boxplot(Xtr_i["duration_min"], vert=False)
ax.set_title("duration_min: long tail is real (kept, not removed)");
plt.tight_layout(); plt.show()

```

	skew	iqr_outliers	min	max
n_directors	9.28	524	0.0	12.0
n_countries	2.95	1047	0.0	12.0
release_year	-2.87	422	1942.0	2021.0
years_to_netflix	2.86	412	0.0	75.0
title_len	1.51	158	1.0	104.0
year_added	-0.67	26	2009.0	2021.0
desc_words	0.66	34	10.0	48.0
n_cast	0.51	35	0.0	44.0
n_genres	-0.27	0	1.0	3.0
duration_min	0.25	286	3.0	312.0
month_added	-0.07	0	1.0	12.0



```

# Normalization = change the SHAPE (reduce skew) with a Yeo-Johnson power transform
skewed = [c for c in num_cols if abs(Xtr_i[c].skew()) > 0.75]
print("Strongly skewed columns normalized:", skewed)

```

```

Xtr_n, Xte_n = Xtr_i.copy(), Xte_i.copy()
if skewed:
    pt = PowerTransformer(method="yeo-johnson", standardize=False).fit(Xtr_i[skewed])
    Xtr_n[skewed] = pt.transform(Xtr_i[skewed])
    Xte_n[skewed] = pt.transform(Xte_i[skewed])

# Scaling = change the RANGE to 0-1 with MinMaxScaler (fit on train or test)
scaler = MinMaxScaler().fit(Xtr_n[num_cols])
Xtr_s, Xte_s = Xtr_n.copy(), Xte_n.copy()
Xtr_s[num_cols] = scaler.transform(Xtr_n[num_cols])
Xte_s[num_cols] = scaler.transform(Xte_n[num_cols])

print("\nSample numeric rows BEFORE normalization/scaling:")
display(Xtr_i[num_cols].head(5).round(2))
print("Sample numeric rows AFTER normalization/scaling:")
display(Xtr_s[num_cols].head(5).round(3))

# Before/after histograms for the most skewed columns
top = skew.abs().sort_values(ascending=False).index[:4].tolist()
fig, axes = plt.subplots(2, len(top), figsize=(4 * len(top), 6))
for j, c in enumerate(top):
    axes[0, j].hist(Xtr_i[c], bins=30, color="tab:blue"); axes[0, j].set_title(c)
    axes[1, j].hist(Xtr_s[c], bins=30, color="tab:green"); axes[1, j].set_title(c)
plt.tight_layout(); plt.show()

```

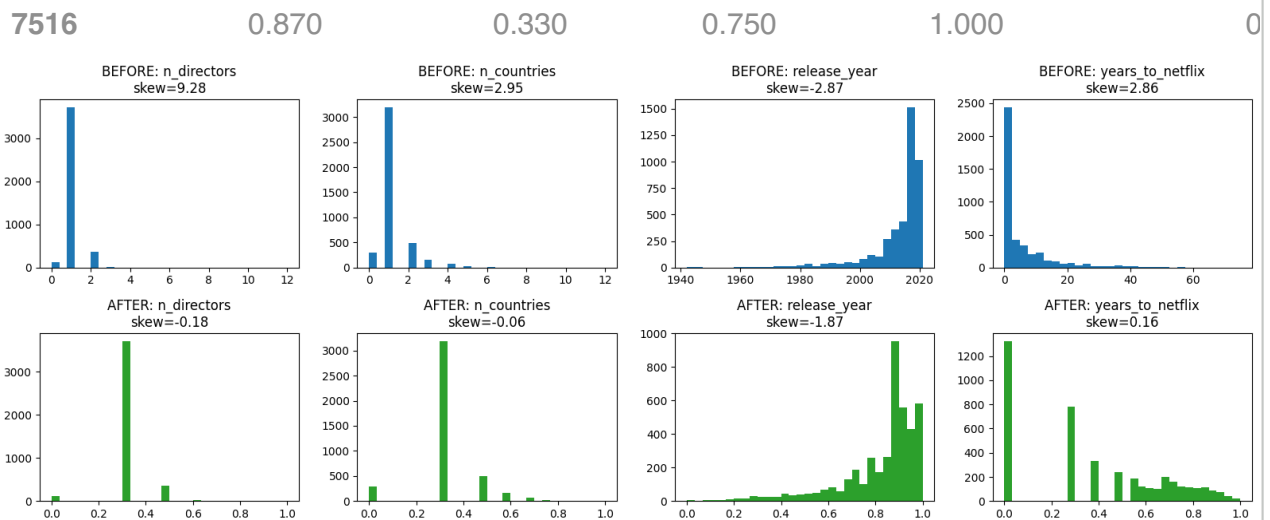
Strongly skewed columns normalized: ['release_year', 'years_to_netflix']

Sample numeric rows BEFORE normalization/scaling:

	release_year	duration_min	year_added	month_added	years_to_netflix
7653	2016	107.0	2018	5	0.000
2646	2016	107.0	2020	4	0.000
3044	2020	102.0	2020	1	0.000
8635	1969	128.0	2020	1	0.000
7516	2016	105.0	2018	12	0.000

Sample numeric rows AFTER normalization/scaling:

	release_year	duration_min	year_added	month_added	years_to_netflix
7653	0.870	0.337	0.750	0.364	0.000
2646	0.870	0.337	0.917	0.273	0.000
3044	0.973	0.320	0.917	0.000	0.000
8635	0.166	0.405	0.917	0.000	0.000



Note: the test set is transformed with the transformers fitted on training data. A few test values can land slightly outside 0 to 1. That is expected and correct for unseen data.

✓ Step 8. Encoding categorical data

```
# (a) One-hot encoding: primary_country, keeping the 10 most common countries
top_countries = Xtr_i["primary_country"].value_counts().head(10).index
for d in (Xtr_i, Xte_i):
    d["country_grouped"] = d["primary_country"].where(d["primary_country"].isin(top_countries), "other")

ohe = OneHotEncoder(handle_unknown="ignore", sparse_output=False).fit(top_countries)
def ohe_frame(d):
    return pd.DataFrame(ohe.transform(d[["country_grouped"]]), index=d.index,
                        columns=[f"country_{c}" for c in ohe.categories_])
```

```

# (b) Multi-hot encoding: a movie can have several genres
mlb = MultiLabelBinarizer().fit(Xtr["genres"])
def genre_frame(d):
    return pd.DataFrame(mlb.transform(d["genres"]), index=d.index, co

# (c) Ordinal encoding: duration bucket has a natural order Short < S
def bucket(d):
    return pd.cut(d["duration_min"], bins=[-np.inf, 90, 120, np.inf],
ordenc = OrdinalEncoder(categories=[["Short", "Standard", "Long"]])
ordenc.fit(bucket(Xtr_i).to_frame("duration_bucket")) # fit on train
def ord_frame(d):
    b = bucket(d).to_frame("duration_bucket")
    return pd.DataFrame(ordenc.transform(b), index=d.index, columns=['

print("Sample categorical rows BEFORE encoding:")
display(pd.concat([Xtr_i[["country_grouped"]], bucket(Xtr_i).rename("c
Xtr["genres"].apply(lambda g: ", ".join(g)).rename

def assemble(d_scaled, d_imputed):
    return pd.concat([d_scaled[num_cols + flag_cols], ohe_frame(d_impu

Xtr_enc = assemble(Xtr_s, Xtr_i) # transformers already fitted
Xte_enc = assemble(Xte_s, Xte_i)

print(f"\nFinal feature matrix: train {Xtr_enc.shape}, test {Xte_enc.s
assert list(Xtr_enc.columns) == list(Xte_enc.columns)
assert Xtr_enc.isna().sum().sum() == 0 and Xte_enc.isna().sum().sum()
print("Sample rows AFTER encoding:")
display(Xtr_enc.iloc[:5, :14].round(3))
print("Encoded column groups: numeric", len(num_cols), "| flags", len
"| country one-hot", len(ohe.categories_[0]), "| genre multi-hot")

```

Sample categorical rows BEFORE encoding:

	country_grouped	duration_bucket	genres
7653	India	Standard	Dramas, International Movies
2646	Other	Standard	Comedies, International Movies
3044	Spain	Standard	Comedies, Dramas, International Movies
8635	United States	Long	Classic Movies, Dramas
7516	United States	Standard	Dramas

Final feature matrix: train (4233, 46), test (1815, 46)
Sample rows AFTER encoding:

	release_year	duration_min	year_added	month_added	years_to_netf
7653	0.870	0.337	0.750	0.364	0
2646	0.870	0.337	0.917	0.273	0
3044	0.973	0.320	0.917	0.000	0
8635	0.166	0.405	0.917	0.000	0
7516	0.870	0.330	0.750	1.000	0

Encoded column groups: numeric 11 | flags 3 | country one-hot 11 | genre

✓ Step 9. Class balancing (training set only)

We first **measure** the real class distribution, then balance only if it is clearly imbalanced. Method: random oversampling of the minority class. This is transparent, works with one-hot data, and never creates fake fractional categories (unlike SMOTE on encoded categories).

```
train = Xtr_enc.copy(); train["kid_safe"] = ytr
counts = train["kid_safe"].value_counts().sort_index()
minority_label = counts.idxmin(); majority_label = counts.idxmax()
ratio = counts.min() / counts.max()
print("Train class counts:\n", counts.rename({0: "mature (0)", 1: "kid_safe (1)"}))
print(f"Minority/majority ratio: {ratio:.2f}")

NEEDS_BALANCING = ratio < 0.8
print("Balancing needed:", NEEDS_BALANCING)

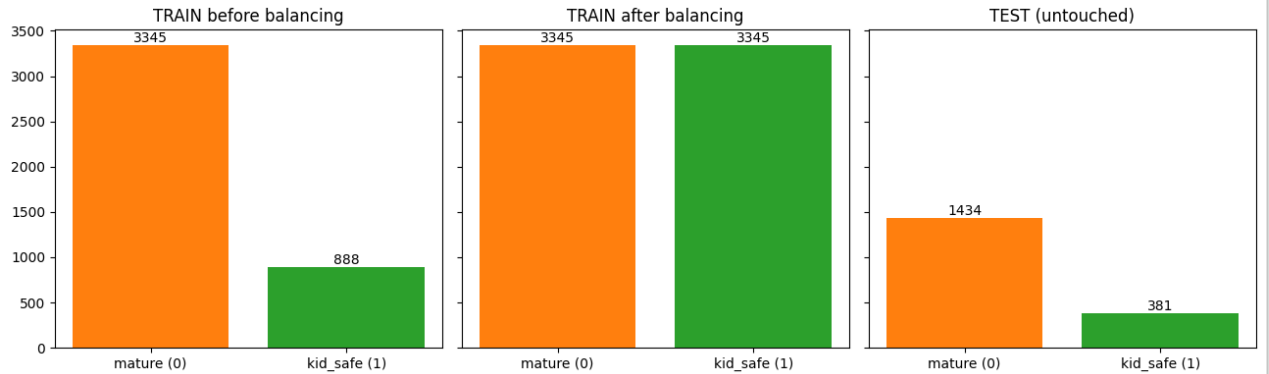
if NEEDS_BALANCING:
    minority = train[train["kid_safe"] == minority_label]
    majority = train[train["kid_safe"] == majority_label]
    minority_up = resample(minority, replace=True, n_samples=len(majority))
    train_bal = pd.concat([majority, minority_up]).sample(frac=1, random_state=42)
else:
    train_bal = train.copy()

fig, axes = plt.subplots(1, 3, figsize=(13, 4), sharey=True)
labels = ["mature (0)", "kid_safe (1)"]
axes[0].bar(labels, train["kid_safe"].value_counts().sort_index().values)
axes[0].set_title("TRAIN before balancing")
axes[1].bar(labels, train_bal["kid_safe"].value_counts().sort_index().values)
axes[1].set_title("TRAIN after balancing")
```

```
axes[2].bar(labels, yte.value_counts().sort_index().values, color=["t", "b"])
axes[2].set_title("TEST (untouched)")
for a in axes:
    for p in a.patches: a.text(p.get_x() + p.get_width() / 2, p.get_height() / 2, str(yte.value_counts().sort_index().values[p.get_x()]))
plt.tight_layout(); plt.show()

print("Sample rows after balancing:")
display(train_bal.iloc[:5, :8].join(train_bal["kid_safe"]).round(3))
```

```
Train class counts:
  kid_safe
mature (0)      3345
kid_safe (1)     888
Name: count, dtype: int64
Minority/majority ratio: 0.27
Balancing needed: True
```



Sample rows after balancing:

	release_year	duration_min	year_added	month_added	years_to_netf
4299	0.921	0.246	0.750	1.000	0
3314	0.823	0.133	0.833	0.909	0
3314	0.823	0.133	0.833	0.909	0
3314	0.823	0.133	0.833	0.909	0
3314	0.823	0.133	0.833	0.909	0
1556	0.973	0.081	0.917	1.000	0
1556	0.973	0.081	0.917	1.000	0
1556	0.973	0.081	0.917	1.000	0
1556	0.973	0.081	0.917	1.000	0
1556	0.973	0.081	0.917	1.000	0
2477	0.946	0.472	0.917	0.364	0
1507	0.973	0.081	0.917	1.000	0

1507	0.973	0.081	0.917	1.000	0
1507	0.973	0.081	0.917	1.000	0
1507	0.973	0.081	0.917	1.000	0
1507	0.973	0.081	0.917	1.000	0
1507	0.973	0.081	0.917	1.000	0

✓ Step 10. Final checks and saved outputs

```

test = Xte_enc.copy(); test["kid_safe"] = yte

assert "rating" not in train_bal.columns
print("Rows: raw", len(raw), "| movies", len(movies), "| modeling", len(modeling), "| train (balanced)", len(train_bal), "| test", len(test))
print("Test kid_safe share (unchanged):", round(test["kid_safe"].mean(), 2))

train.to_csv("netflix_movies_train_unbalanced.csv", index=False)
train_bal.to_csv("netflix_movies_train_balanced.csv", index=False)
test.to_csv("netflix_movies_test.csv", index=False)
print("Saved: netflix_movies_train_unbalanced.csv, netflix_movies_train_balanced.csv, netflix_movies_test.csv")

# Optional in Colab: download the files
try:
    from google.colab import files
    for f in ["netflix_movies_train_balanced.csv", "netflix_movies_test.csv"]:
        files.download(f)
except ImportError:
    pass

```

```

Rows: raw 8807 | movies 6131 | modeling 6048 | train (balanced) 6690 | test 1117
Test kid_safe share (unchanged): 0.21
Saved: netflix_movies_train_unbalanced.csv, netflix_movies_train_balanced.csv, netflix_movies_test.csv

```

Step 11. How this dataset solves an ML problem

Problem: Given a Netflix movie's metadata (genres, country, run time, release year, when it was added, cast and director counts, description length), predict whether it is **kid-safe** (`kid_safe = 1`) or mature (`0`) using Binary Classification.

Why it is useful: A streaming service like Netflix could use the model to flag likely kid-friendly titles for a kids profile, or to suggest a maturity category for titles that arrive with no rating.

Why the target is not a leak: The `rating` column is used only to *create* the label and is removed from the inputs, so models must learn from content features instead.

Planned models for the finals: Logistic regression (baseline), k-nearest neighbors, decision tree, random forest, gradient boosting.

Evaluation plan: Because kid-safe movies are by far the minority, accuracy alone can look good while missing them. We will report precision, recall, F1, and ROC-AUC on the untouched 30% test set, and review the confusion matrix. For this use case, labeling a mature movie as kid-safe (false positive) is the costlier mistake, so precision on `kid_safe` matters.

Known limits: Genre tags such as "Children & Family Movies" are very informative, so models may lean on them heavily. Unrated movies were excluded, so the model has not seen them. The text `description` is not yet used; TF-IDF features are an option for the Finals.

